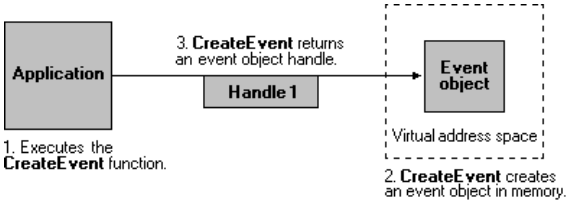
Kernel Objects

Kernel object handles are process specific. That is, a process must either create the object or open an existing object to obtain a kernel object handle. The per-process limit on kernel handles is 2^24. However, handles are stored in the paged pool, so the actual number of handles you can create is based on available memory. The number of handles that you can create on 32-bit Windows is significantly lower than 2^24.

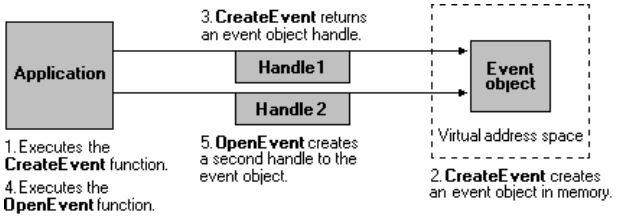
Any process can create a new handle to an existing kernel object (even one created by another process), provided that the process knows the name of the object and has security access to the object. Kernel object handles include access rights that indicate the actions that can be granted or denied to a process. An application specifies access rights when it creates an object or obtains an existing object handle. Each type of kernel object supports its own set of access rights. For example, event handles can have set or wait access (or both), file handles can have read or write access (or both), and so on. For more information, see Securable Objects.

In the following illustration, an application creates an event object. The **CreateEvent** function creates the event object and returns an object handle.



After the event object has been created, the application can use the event handle to set or wait on the event. The handle remains valid until the application closes the handle or terminates.

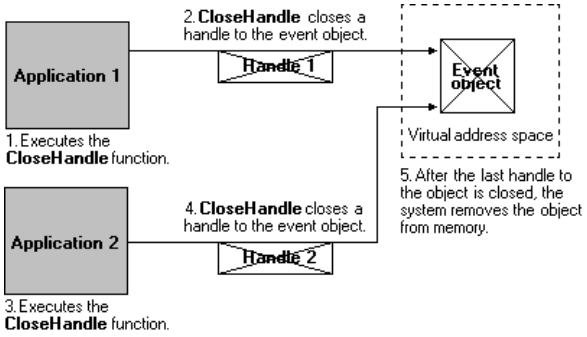
Most kernel objects support multiple handles to a single object. For example, the application in the preceding illustration could obtain additional event object handles by using the **OpenEvent** function, as shown in the following illustration.



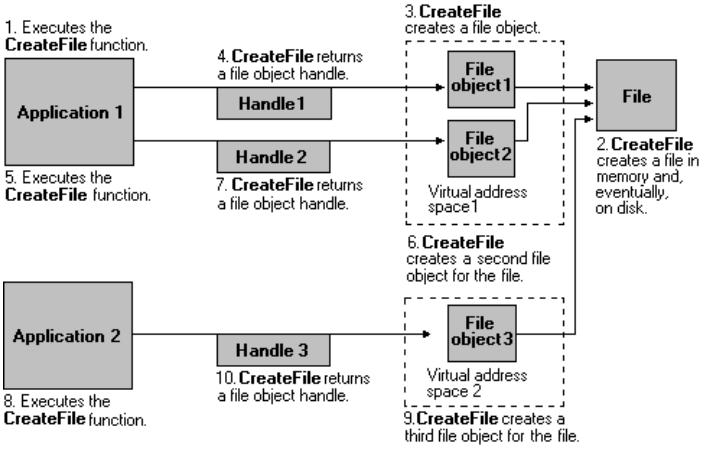
This method enables an application to have handles with different access rights. For example, Handle 1 might have set and wait access to the event, and Handle 2 might have only wait access.

If another process knows the event name and has security access to the object, it can create its own event object handle by using **OpenEvent**. The creating application could also duplicate one of its handles into the same process or into another process by using the **DuplicateHandle** function.

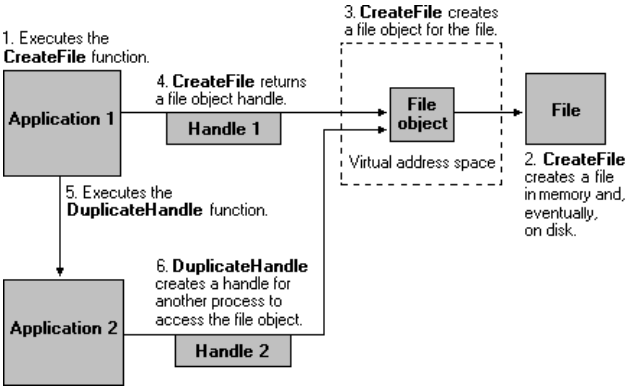
An object remains in memory as long as at least one object handle exists. In the following illustration, the applications use the **CloseHandle** function to close their event object handles. When there are no event handles, the system removes the object from memory, as shown in the following illustration.



The system manages file objects somewhat differently from other kernel objects. File objects contain the file pointer — the pointer to the next byte to be read or written in a file. Whenever an application creates a new file handle, the system creates a new file object. Therefore, more than one file object can refer to a single file on disk, as shown in the next illustration.



Only through duplication or inheritance can more than one file handle refer to the same file object, as shown in the following illustration.



The following table lists each of the kernel objects, along with each object's creator and destroyer functions. The creator functions either create the object and an object handle or create a new existing object handle. The destroyer functions close the object handle. When an application closes the last handle to a kernel object, the system removes the object from memory.

Kernel object	Creator function	Destroyer function
Access token	**CreateRestrictedToken**, **DuplicateToken**, **DuplicateTokenEx**, **OpenProcessToken**, **OpenThreadToken**	**CloseHandle**
Change notification	**FindFirstChangeNotification**	**FindCloseChangeNotification**
Communications device	**CreateFile**	**CloseHandle**
Console input	**CreateFile**, with CONIN$	**CloseHandle**
Console screen buffer	**CreateFile**, with CONOUT$	**CloseHandle**
Desktop	**GetThreadDesktop**	Applications cannot delete this object.
Event	**CreateEvent**, **CreateEventEx**, **OpenEvent**	**CloseHandle**
Event log	**OpenEventLog**, **RegisterEventSource**, **OpenBackupEventLog**	**CloseEventLog**
File	**CreateFile**	**CloseHandle**, **DeleteFile**
File mapping	**CreateFileMapping**, **OpenFileMapping**	**CloseHandle**
Find file	**FindFirstFile**	**FindClose**
Heap	**HeapCreate**	**HeapDestroy**
I/O completion port	**CreateIoCompletionPort**	**CloseHandle**
Job	**CreateJobObject**	**CloseHandle**
Mailslot	**CreateMailslot**	**CloseHandle**

Memory resource notification	**CreateMemoryResourceNotification**	**CloseHandle**
Module	**LoadLibrary**, **GetModuleHandle**	**FreeLibrary**
Mutex	**CreateMutex**, **CreateMutexEx**, **OpenMutex**	**CloseHandle**
Pipe	**CreateNamedPipe**, **CreatePipe**	**CloseHandle**, **DisconnectNamedPipe**
Process	**CreateProcess**, **OpenProcess**, **GetCurrentProcess**	**CloseHandle**, **TerminateProcess**
Semaphore	**CreateSemaphore**, **CreateSemaphoreEx**, **OpenSemaphore**	**CloseHandle**
Socket	**socket**, **accept**	**closesocket**
Thread	**CreateThread**, **CreateRemoteThread**, **GetCurrentThread**	**CloseHandle**, **TerminateThread**
Timer	**CreateWaitableTimer**, **CreateWaitableTimerEx**, **OpenWaitableTimer**	**CloseHandle**
Update resource	**BeginUpdateResource**	**EndUpdateResource**
Window station	**GetProcessWindowStation**	Applications cannot delete this object.

Related topics

Kernel Object Namespaces

Community Additions

desktop

DesktopGetThreadDesktopApplications cannot delete this object.
为何这样说呢？请看下面的链接。

而且desktops.exe这个工具也没有销毁桌面的功能。

望给予解释。

顺便还有：**CloseWindowStation**。

进一步的还有：

win32k!NtUserCloseDesktop

win32k!NtUserCloseWindowStation

CloseDesktop

https://msdn.microsoft.com/en-us/library/windows/desktop/ms682024%28v=vs.85%29.aspx

kouleguan
4/23/2015

Is that all of the KObjs?

In Windows Kernel Programming Core 5th, there's RegOpenKeyEx, however, it is not listed here.

wangwentao
4/15/2015

遗漏了？还是不完整？

https://msdn.microsoft.com/en-us/library/windows/hardware/ff565763%28v=vs.85%29.aspx
里有：
Registry keys
等等。

https://msdn.microsoft.com/zh-cn/library/ff554383%28v=vs.85%29.aspx
里有：
Registry keys.
等等。

是不是遗漏了？这里写的比较全面，感觉还不是最全。

kouleguan
3/16/2015

Handle object types

The question is what the object categories are numerically. When enumerating handles, I see for example that object type 8 is a thread, 22 = TpWorkerFactory etc. Is there a full list of this?

Ruud van Gaal
5/15/2014

doubt

Is "Module" really a Kernel Object? or only a base address?
I have read some codes of the implemention of the LoadLibrary and GetModuleHanle of the ReactOS
but got nothing to prove that Module is a KO!!!
Who can help me....Thanks~

doubt!!
12/21/2009

Incomplete list?

For example, what about HttpCreateRequestQueue ?

http://msdn.microsoft.com/en-us/library/aa364483%28VS.85%29.aspx

fowl
10/17/2009

© 2015 Microsoft